

AIxRWA

Strategic RWA Allocation & Smart Yield Optimisation

The Definitive Strategy Guide

Mantle Turing Test Hackathon 2026 · AI × RWA Track

Focus Assets USDY + mETH	Target Chain Mantle L2
Strategies Covered 5 Core Frameworks	Version May 2026

01. Foundations — What RWA and Smart Allocation Mean

Reading guide

This document defines five allocation strategies for managing USDY and mETH on Mantle. Each strategy is explained in plain language, followed by step-by-step methodology, exact formulas, a pros/cons evaluation, and implementation code. Read top-to-bottom for a complete understanding, or jump to any strategy using the section numbers.

1.1 What Is RWA Allocation?

In the context of YieldMind AI, 'RWA allocation' means deciding what percentage of the managed portfolio to hold in each real-world asset at any given time. The two assets are:

USDY — Ondo US Dollar Yield

Backed by US Treasury bills. Yields approximately 4.8–5.2% APY. Price appreciates daily (non-rebasing). Denominated in USD — value is predictable, low volatility.

Risk profile: Very low. The only realistic risks are Ondo protocol failure or a US Treasury default — both extremely unlikely.

mETH — Mantle Staked Ether

Backed by ETH staking rewards. Yields approximately 3.8% APY in ETH terms. Exchange rate to ETH rises continuously. Denominated in ETH — USD value fluctuates with ETH price.

Risk profile: Low-medium. Staking rewards are steady; the risk is ETH price volatility compressing USD-denominated yield.

1.2 What Is Smart Allocation?

Smart allocation means the portfolio does not stay at a fixed split forever. It rebalances in response to three types of signals:

Yield signal

When USDY APY is meaningfully higher than mETH APY (in USD terms), shift weight towards USDY. When mETH yield is higher, shift towards mETH.

Risk signal

When a risk threshold is breached — USDY depeg, ETH price drop, oracle staleness — shift weight toward the safer asset or hold cash.

Efficiency signal

When the yield difference between assets is too small to justify swap costs (gas + slippage), hold current positions. Never swap just for the sake of moving.

02. Strategy 1 — Static Balanced Allocation

S1 The Baseline — Hold 50/50 and rebalance monthly

The simplest possible strategy. Hold exactly 50% USDY and 50% mETH at all times. Rebalance back to 50/50 on a fixed monthly schedule. No AI required. This is the benchmark that all smarter strategies must outperform.

2.1 How It Works — Step by Step

1. Deposit total capital into the protocol. Split exactly 50% into USDY, 50% into mETH at current market prices.
2. On day 1 of every month, query balanceOf() for both assets and convert both to USD using current prices.
3. If the USDY side is now worth more than 52% of total (due to price changes), sell the excess into mETH. If mETH side is above 52%, sell excess into USDY.
4. The 2% tolerance band (48%–52%) prevents excessive rebalancing from tiny daily fluctuations.
5. Log the rebalancing transaction on-chain with timestamp and amounts.

2.2 The Core Formula

Rebalance trigger:

$$\text{Rebalance needed} = |\text{USDY_USD} / \text{Total_USD} - 0.50| > 0.02$$

Swap amount:

$$\text{Swap_amount} = (\text{USDY_USD} - \text{Total_USD} \times 0.50)$$

Example: Portfolio = \$10,000 total. USDY = \$5,800, mETH = \$4,200. USDY weight = 58% > 52% threshold. Swap \$800 of USDY into mETH to return to 50/50.

2.3 Pros and Cons

Advantages	Limitations
- Extremely simple to implement — no AI needed for this layer - Natural diversification between USD-stable and ETH-correlated yield - Monthly rebalancing is low-cost on Mantle (~\$0.002 per swap)	- Leaves yield on the table when one asset clearly offers better returns - Does not respond to risk events (USDY depeg, ETH crash) until the monthly rebalance

- | | |
|---|---|
| <ul style="list-style-type: none">• Serves as the perfect benchmark: judges can compare AI strategy vs this baseline• Easy to explain and audit — every action is predictable• Zero risk of AI hallucination or bad decisions | <ul style="list-style-type: none">• No adaptation to changing yield differentials — ignores real-time signals• Monthly cadence may be too slow when yield opportunities are short-lived• Generates roughly the average of both yields, never the best of either |
|---|---|

03. Strategy 2 — Dynamic Yield-Differential Allocation

S2

The Core Strategy — Shift weight toward the higher-yielding asset

The primary strategy of YieldMind AI. The agent continuously monitors the APY of both assets in USD terms and tilts the portfolio toward whichever is earning more — but only when the yield difference is large enough to justify the swap cost. This is where the AI adds measurable value over a static baseline.

3.1 How It Works — Step by Step

6. Every 2 hours, the agent polls the USDY oracle for current USD price and the mETH staking contract for the current ETH:mETH exchange rate.
7. Convert mETH yield to USD terms: multiply the ETH-denominated APY by the current ETH/USD price.
8. Calculate the net yield differential: `usdy_apy_usd` minus `meth_apy_usd`. A positive number means USDY is outperforming.
9. Subtract estimated swap cost (slippage + gas, typically 0.3% on Mantle) from the differential.
10. If the net differential exceeds the minimum threshold (0.5% APY), the agent initiates a rebalancing swap. The swap size is proportional to the differential magnitude.
11. If the differential is below threshold, the agent holds. 'Hold' is always the safe default.

3.2 The Allocation Formula

Step 1 — Convert mETH yield to USD:

$$\text{mETH_apy_usd} = \text{mETH_apy_eth} \times \text{ETH_price_usd} / 100$$

Step 2 — Net differential after costs:

$$\text{net_diff} = |\text{usdy_apy_usd} - \text{mETH_apy_usd}| - \text{swap_cost}$$

Step 3 — Target weight for leading asset:

$$\text{target_weight} = 0.50 + \min(\text{net_diff} \times 10, 0.25)$$

Cap: maximum 75% in any single asset (50% base + 25% tilt). This prevents over-concentration in one asset, regardless of yield difference.

Example: USDY at 5.2%, mETH at 3.8% (USD-adjusted), swap cost 0.3%. Net diff = 1.4% - 0.3% = 1.1%. Target USDY weight = 50% + min(1.1% × 10, 25%) = 50% + 11% = 61%.

3.3 Implementation Code

```
def compute_target_allocation(usdy_apy: float, meth_apy_eth: float,
                             eth_price: float, swap_cost: float = 0.003) -> dict:
    # Convert mETH yield to USD-denominated APY
    meth_apy_usd = (meth_apy_eth / 100) * eth_price
    usdy_apy_dec = usdy_apy / 100

    # Net yield advantage after estimated swap cost
    raw_diff = usdy_apy_dec - meth_apy_usd
    net_diff = abs(raw_diff) - swap_cost

    # Below minimum threshold — not worth swapping
    MIN_THRESHOLD = 0.005 # 0.5% APY minimum advantage
    if net_diff < MIN_THRESHOLD:
        return {'action': 'hold', 'usdy_weight': None, 'reason':
            'diff_below_threshold'}

    # Proportional tilt: 10x multiplier, capped at 25% extra weight
    tilt = min(net_diff * 10, 0.25)
    if raw_diff > 0: # USDY is leading
        return {'action': 'rebalance', 'usdy_weight': 0.50 + tilt,
            'meth_weight': 0.50 - tilt, 'leading': 'USDY'}
    else: # mETH is leading
        return {'action': 'rebalance', 'usdy_weight': 0.50 - tilt,
            'meth_weight': 0.50 + tilt, 'leading': 'mETH'}
```

3.4 Worked Example

Scenario	USDY APY	mETH APY (USD)	Net Differential	Agent Action
High USDY	5.2%	3.8%	1.1% (after cost)	61% USDY, 39% mETH
Near-equal	4.5%	4.3%	-0.1% (below min)	HOLD — no swap
High mETH (ETH rally)	4.8%	6.1%	0.8% (after cost)	42% USDY, 58% mETH
Max tilt (rare)	5.8%	2.9%	2.6% (after cost)	75% USDY, 25% mETH

Advantages	Limitations
- Captures yield upside when one asset clearly outperforms - Proportional sizing prevents over-reaction to small differences	- ETH price volatility can make mETH appear to outperform when it is actually compressing in real terms - Two-hour polling frequency may miss short-lived yield spikes

- | | |
|---|---|
| <ul style="list-style-type: none">• Net-of-cost calculation ensures every swap is profitable on paper• Swap cost threshold eliminates spurious trades — agent holds when uncertain• Directly demonstrable: show judges the agent earning more than the 50/50 baseline | <ul style="list-style-type: none">• Yield differential alone ignores liquidity risk and pool depth• Performance depends on the accuracy of ETH price oracle feed• Maximum 25% tilt means the agent can never go all-in, even when one asset is clearly dominant |
|---|---|

04. Strategy 3 — Risk-Weighted Allocation

S3

The Safety Layer — Adjust position size based on risk level

While Strategy 2 optimises for yield, Strategy 3 ensures the agent's position sizing reflects the risk environment at all times. A high-risk environment reduces all positions and moves weight toward the safer asset, regardless of yield signals. This is the strategy layer that protects capital during adverse market conditions.

4.1 The Risk Score System

The agent maintains a composite risk score from 0 (no risk) to 100 (maximum risk) computed from five independent signals:

Signal	What It Measures	Score Range	Trigger Level
USDY Price Deviation	Distance of USDY from \$1.00 peg	0–40	Score 40 if price < \$0.990
mETH Exchange Rate Drop	Rate drop vs 24h ago in % terms	0–30	Score 30 if rate drops > 0.3%
Oracle Staleness	Hours since last oracle update	0–15	Score 15 if update > 25 hours ago
DEX Liquidity Depth	Available liquidity at current price	0–10	Score 10 if depth < \$100k
Gas Spike Factor	Current gas vs 30-day average	0–5	Score 5 if gas > 5× average

4.2 Risk Score → Position Sizing

Risk Score	Risk Level	Max Position Size	USDY Floor	Action
0–20	Green — Normal	75% any asset	30%	Normal S2 rules apply
21–45	Yellow — Caution	60% any asset	40%	Reduce swap size by 50%
46–70	Orange — Elevated	45% any asset	55%	Only defensive

				moves allowed
71–89	Red — High Risk	25% any asset	75%	Shift toward USDY only
90–100	Black — Emergency	0% active	100%	HALT — emergency exit to USDY

4.3 Risk Score Calculation Code

```
def compute_risk_score(market_data: dict) -> int:
    score = 0

    # Signal 1 — USDY Peg (max 40 pts)
    usdy_price = market_data['usdy_price']
    if usdy_price < 0.990: score += 40
    elif usdy_price < 0.995: score += 20
    elif usdy_price < 0.998: score += 8

    # Signal 2 — mETH Rate Drop (max 30 pts)
    rate_change = market_data['meth_rate_24h_change']
    if rate_change < -0.003: score += 30
    elif rate_change < -0.001: score += 15

    # Signal 3 — Oracle Staleness (max 15 pts)
    hours_stale = market_data['usdy_oracle_hours_since_update']
    if hours_stale > 26: score += 15
    elif hours_stale > 24: score += 8

    # Signal 4 — DEX Liquidity (max 10 pts)
    liquidity_usd = market_data['pool_liquidity_usd']
    if liquidity_usd < 50_000: score += 10
    elif liquidity_usd < 100_000: score += 5

    # Signal 5 — Gas Spike (max 5 pts)
    gas_multiplier = market_data['gas_vs_30d_average']
    if gas_multiplier > 5: score += 5

    return min(score, 100)
```

Advantages	Limitations
Capital preservation during market stress — automatic de-risking before losses occur	Five signals require five data sources — polling overhead increases agent complexity

- | | |
|---|---|
| <ul style="list-style-type: none">• Five independent signals ensure no single bad data point triggers false alarms• Graduated response: risk level 2 caution is very different from level 5 emergency• On-chain logging of all risk score changes creates an auditable safety history• Directly addresses judges' risk management questions with a quantified, transparent framework | <ul style="list-style-type: none">• Risk score is additive, not probabilistic — may overweight correlated events• USDY floor at high risk levels may lock the agent out of mETH yield during temporary spikes• Emergency exit to USDY burns swap costs even if the risk event resolves quickly• Calibration of score thresholds requires backtesting — parameters are judgment calls initially |
|---|---|

05. Strategy 4 — Confidence-Weighted AI Allocation

S4

The Intelligence Layer — Let the LLM vote, but weight by confidence

This strategy adds the LLM reasoning layer on top of the quantitative signals from Strategies 2 and 3. The AI agent receives a structured prompt containing current yield data, risk scores, recent price history, and market context — and returns a decision with a confidence score. The final position is a weighted combination of the quantitative model and the AI recommendation.

5.1 How the AI Vote Works

The LLM does not replace the quantitative model — it augments it. Think of it as a second opinion with a stated confidence level. The system only acts on AI recommendations when confidence exceeds 0.65 and the quantitative model also agrees directionally.

System prompt structure:

The prompt gives the AI: (1) current USDY and mETH APYs, (2) 7-day yield trend for each, (3) current risk score and which signals are elevated, (4) last 3 agent decisions and their outcomes, (5) current portfolio weights. The AI returns a JSON object with action, recommended weight, reasoning, and confidence.

Confidence gating rule:

If AI confidence < 0.65: ignore AI vote entirely, follow quantitative model only. If confidence >= 0.65: blend AI recommendation with quantitative target (70% quant / 30% AI). If confidence >= 0.85 AND quant model agrees directionally: blend 50/50.

5.2 Final Allocation Formula

```
def compute_final_allocation(quant_target: float, ai_decision: dict,
                             risk_score: int) -> float:
    '''Returns target USDY weight (0.25 to 0.75).'''

    confidence = ai_decision.get('confidence', 0.0)
    ai_weight = ai_decision.get('usdy_weight', quant_target)

    # AI gating - ignore low-confidence recommendations
    if confidence < 0.65:
        final = quant_target # Pure quantitative
    elif confidence >= 0.85 and same_direction(quant_target, ai_weight):
        final = 0.5 * quant_target + 0.5 * ai_weight # 50/50 blend
    else:
        final = 0.7 * quant_target + 0.3 * ai_weight # 70/30 blend
```

```
# Apply risk score cap — risk overrides everything
max_weight = get_max_weight_from_risk(risk_score) # from Strategy 3
final = min(final, max_weight)
final = max(final, get_min_usdy_floor(risk_score))

return round(final, 4) # 4 decimal precision

def same_direction(quant: float, ai: float, base: float = 0.5) -> bool:
    return (quant - base) * (ai - base) > 0 # Both above or both below 50%
```

Advantages	Limitations
- Combines quantitative rigour with contextual AI reasoning — neither alone is as good - Confidence gating prevents hallucinated decisions from affecting the portfolio 70/30 blend means bad AI calls have limited impact (max 7.5% position difference) - AI reasoning in the log explains non-obvious decisions to judges and users - Self-improving: log correct and incorrect decisions to improve future prompts - Gracefully degrades — if AI is down or slow, pure quantitative mode takes over	- Adds LLM inference cost and latency to every decision cycle - Model quality is non-deterministic — same market conditions may yield different answers - Prompt engineering quality significantly affects output quality — requires iteration - Blending two models introduces complexity in attributing performance outcomes - Requires testing on historical data to calibrate confidence threshold (0.65 is a starting point) - AI may be confidently wrong — high confidence does not guarantee accuracy

06. Strategy 5 — Aave Horizon Collateral Layer

S5

The Advanced Layer — Borrow against RWAs to unlock triple yield

This is the highest-complexity strategy and is planned for YieldMind AI version 2. It uses Aave Horizon — a permissioned Aave market on Mantle with \$1.34B in lending activity — to convert held RWA positions into productive collateral. Instead of just holding USDY for 5.2% APY, the agent deposits USDY as collateral, borrows USDC against it at a lower rate, and deploys that USDC into another yield opportunity — earning yield on both the original position and the borrowed capital.

6.1 The Three-Layer Yield Stack

#	Layer	How It Earns	Example Annual Yield
1	USDY Base Yield	Hold USDY. US Treasury yield accrues into token price daily.	5.2% on 100% of position
2	Collateral Borrow Spread	Deposit USDY to Aave Horizon. Borrow USDC at 3.5% APR against 60% LTV. The spread = USDY yield minus borrow cost.	1.7% spread on 60% of position = +1.02%
3	Borrowed Capital Yield	Deploy borrowed USDC into highest available yield (e.g. mETH purchase, or Aave supply).	~3.8% on 60% = +2.28%
Σ	Total blended yield	5.2% + 1.02% + 2.28%	8.5% vs 5.2% base

6.2 Critical Safety Rule — Health Factor

The most important number in the collateral strategy is the Aave Health Factor.

Health Factor = (Collateral Value × Liquidation Threshold) / Borrowed Value. If this falls below 1.0, Aave liquidates your position at a penalty, potentially wiping out gains.

Safe operating rule: never let Health Factor fall below 1.5. At 60% LTV and 85% liquidation threshold, USDY price must drop 29% before liquidation risk. Always maintain a buffer.

```
Health Factor = (USDY_balance × usdy_price × 0.85) / borrowed_usdc
```

Emergency rule: If Health Factor < 1.5, repay 30% of borrowed USDC immediately.

Advantages	Limitations
• Theoretical yield uplift of 60%+ over base strategy (5.2% to 8.5%) with same capital • Leverages Mantle's existing Aave Horizon infrastructure — no new protocol needed • Borrowed capital generates yield rather than sitting idle • Non-liquidating as long as Health Factor stays above 1.5 — safety buffer is large for USDY • Demonstrates sophisticated DeFi composability that impresses technical judges	• Most complex strategy — requires three contract interactions per rebalancing cycle • Liquidation risk exists, even if low — must be actively monitored every cycle • USDY price drop (e.g. temporary depeg) can rapidly compress Health Factor • Borrow rate on Aave can increase, compressing the spread and making the strategy unprofitable • Not recommended for hackathon MVP — implement in version 2 after core strategy is stable

07. The Unified Decision Framework

All five strategies combine into a single decision pipeline. Every time the agent runs, it executes this sequence — from raw data collection to final on-chain action. No step is skipped.

Step	Layer	What Happens	Output
1	Data	Poll USDY oracle price, mETH exchange rate, ETH/USD price, pool liquidity depth, oracle update timestamp.	<code>Raw market_data dict</code>
2	S3 Risk	Compute risk score from 5 signals. If score ≥ 90 , skip all further steps and trigger emergency exit to USDY.	<code>risk_score (0-100)</code>
3	S2 Yield	Compute net yield differential. If below 0.5% threshold, action = 'hold'. Otherwise, compute quant target weight.	<code>quant_target_weight</code>
4	S4 AI	If action $\neq$ 'hold', call LLM with structured prompt. Parse JSON response. Apply confidence gate (0.65 threshold).	<code>ai_decision + confidence</code>
5	Blend	Compute final_weight by blending quant_target and ai_decision using confidence-weighted formula from Strategy 4.	<code>final_usdy_weight</code>
6	Execute	If final_weight differs from current_weight by more than 2%, compute swap amount and call Solidity executor.	<code>tx_hash or null</code>
7	Log	Write complete decision record to JSONL log: all inputs, intermediate values, final decision, tx hash.	<code>agent_log.jsonl</code>

7.1 Strategy Comparison Summary

Strategy	Complexity	Expected Yield	Risk Control	MVP Priority	Best For
S1 Static	★☆☆☆☆	~4.5% avg	Passive	Benchmark	Baseline comparison
S2 Yield Diff	★★★★☆	4.8–5.5%	Moderate	Week 2	Core agent logic

S3 Risk-Weighted	★★★★☆	4.2–5.5%	High	Week 2	Safety layer
S4 AI Confidence	★★★★☆	5.0–5.8%	High	Week 3	Differentiation
S5 Collateral	★★★★★	7.0–9.0%	Very High	v2.0 only	Future version

7.2 What Judges Will Ask — and How to Answer

Q: "How does your agent decide when to trade?"

A: "It runs five risk checks first. If risk is normal, it computes the net yield advantage of USDY over mETH after swap costs. If the advantage exceeds 0.5% APY, it asks the LLM for a recommendation. It only acts if both the quantitative model and the AI agree directionally."

Q: "What stops it from losing money?"

A: "Three independent layers: (1) minimum yield differential threshold means every swap is net-profitable on paper before execution, (2) the risk score auto-reduces position sizes when market conditions deteriorate, (3) a hard-coded 25% maximum position change per trade limits exposure from any single decision."

Q: "How do you prove the AI actually improves performance?"

A: "We run Strategy 1 (static 50/50) in parallel as a benchmark. The live dashboard shows cumulative PnL for both strategies from the same starting capital. The on-chain decision log provides a permanent, auditable record of every AI-made decision and its actual outcome."